Escuela de Ingeniería Informática

Sistemas Operativos

Informe Proyecto - Fase I

Christian Vilera

Noviembre de 2026

1 Arquitectura y concurrencia

Archivo o módulo	Responsabilidad
src/main.cpp	Inicia los hilos, atiende el ciclo del monitor y coordina el cierre. La opción interna --gui-monitor entra directamente al proceso Qt.
src/runtime.hpp y runtime.cpp	Poseen el modelo y serializan las operaciones. Crean el hilo motor y exponen instantáneas para las vistas.
src/model.hpp y model.cpp	Definen PCB, segmentos, reloj, cola de listos y transiciones. Permiten probar la lógica con llamadas directas a step().
src/cli.hpp y cli.cpp	Mantienen el contexto Shell, interpretan comandos, leen la traza y presentan resultados o errores.
src/gui.hpp y gui.cpp	Implementan la interfaz del monitor, su vista Qt, el proceso auxiliar y el transporte de instantáneas.

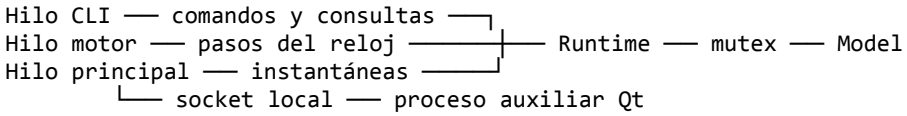


Figura 1. Hilos del simulador y flujo de datos hacia el monitor opcional.

El hilo principal crea el motor por medio de Runtime::launch y después crea la CLI con pthread_create. El objeto Lock adquiere pthread_mutex_t al entrar a una operación y lo libera al salir del ámbito, también cuando se propaga una excepción. Bajo el cerrojo se ejecutan cambios de política, altas, confirmaciones de lotes, pasos y copias del modelo. Las impresiones, la lectura del CSV, la serialización de Qt y los retardos reales ocurren fuera del cerrojo.

Mientras la simulación está pausada, el motor espera en pthread_cond_wait dentro de un bucle que reevalúa el predicado. La espera libera el mutex y lo readquiere al despertar. En marcha, el motor toma la velocidad y la generación del intervalo, espera fuera del mutex en tramos de hasta 10 ms y comprueba de nuevo la generación antes de consumir el tick. La pausa o el cierre invalidan un intervalo anterior. La velocidad nueva se lee al comenzar un intervalo.

Con exit o fin de entrada, la CLI solicita el cierre y publica finished como bandera atómica. El hilo principal abandona el ciclo del monitor, une la CLI y el motor con pthread_join y propaga el estado de salida. Runtime libera la condición y el mutex tras unir el motor.

2 PCB y políticas de planificación

Cada PCB guarda PID, nombre, prioridad, ráfaga inicial y restante, llegada, espera acumulada, primera ejecución y finalización. El PID se asigna desde 1 en orden creciente. Los dos últimos tiempos de evento se representan con valores opcionales. El modelo almacena PCB en un mapa por PID, mantiene una cola de listos y representa la CPU con un PID actual opcional.

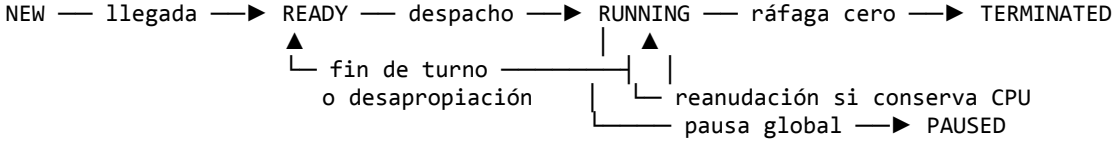


Figura 2. Ciclo de vida. Al reanudar, SRTF puede devolver el proceso pausado a READY.

En create, NEW es transitorio. Un PCB de lote con llegada futura permanece en NEW hasta su tick de admisión. PAUSED representa el proceso que ocupaba la CPU al congelarse el reloj.

Orden de un tick

En la frontera t se admiten llegadas y se despacha. Para el intervalo [t, t+1), el motor suma una unidad de espera a cada PID listo, consume una unidad de ráfaga si hay un proceso en CPU y registra el intervalo. La primera ejecución queda registrada al consumir el primer tick. Tras avanzar el reloj, se admiten las llegadas en t+1, se resuelve la finalización o el vencimiento del quantum y se despacha para el próximo intervalo. Un PCB puede mostrar RUNNING antes de tener tiempo de respuesta registrado.

Round Robin

RR toma el primer PID de la cola FIFO. Si el quantum vence con ráfaga pendiente, agrega el proceso al final de la cola. La finalización tiene precedencia cuando coincide con el vencimiento. Una llegada en la misma frontera entra antes del PID desalojado. El quantum inicial es de 2 ticks y puede configurarse con un entero positivo.

Shortest Remaining Time First

SRTF escoge entre listos por la tupla (ráfaga restante, llegada, PID). Solo reemplaza al proceso actual si el candidato tiene una ráfaga restante estrictamente menor. La prioridad se conserva para inspección y queda fuera de los criterios de RR y SRTF. El cambio de política conserva procesos y colas, reinicia el contador de turno y reevalúa el despacho si el motor está activo. Si está pausado, la reevaluación se produce al reanudar.

3 Consola y procesamiento por lotes

Comando	Función y validación
create <nombre> <burst> [prioridad]	Nombre sin espacios, ráfaga entera positiva y prioridad de 1 a 5. Valor por defecto 3.
load <archivo>	Lee una ruta como un argumento y valida el CSV antes de confirmar la carga.
ps · pcb <PID>	Lista los PCB o detalla uno. El PID debe ser positivo y existir.
set_sched RR [quantum] · set_sched SRTF	Acepta los identificadores exactos. RR admite quantum positivo y SRTF carece de ese argumento.
set_speed <ms>	Admite de 50 a 5000 ms por tick. Inicio en 1000 ms.
start · pause	Reanuda o congela el reloj. El simulador arranca pausado.
gantt · stats	Muestran historial de CPU y estadísticas de la instantánea.
gui · help · exit	Solicita el monitor, muestra ayuda o inicia el cierre. EOF también cierra.

La CLI divide la línea por espacios y valida aridad antes de operar. La conversión por from_chars exige consumir todo el token, y distingue un entero inválido de uno fuera del rango del comando. Así, pcb xyz falla por tipo, create BadProcess por aridad y set_sched FIFO por política inválida. Las consultas presentan una copia coherente tomada bajo mutex.

Carga de la traza

El lector admite cuatro campos por fila en el orden nombre, ráfaga, prioridad y llegada. Elimina espacios alrededor de campos y omite líneas vacías o comentarios que comienzan con #. Rechaza archivos vacíos, incompletos, ilegibles o con valores fuera de rango. El modelo ensaya la carga sobre una copia y publica el lote completo bajo mutex. Una llegada anterior al reloj en el momento de confirmar invalida la operación sin consumir PID. El orden del archivo determina los PID aunque las llegadas estén desordenadas.

Proceso	Ráfaga	Prioridad	Llegada
Job_A	4	2	0
Job_B	2	1	1
Job_C	1	3	3

Tabla. Traza distribuida en tests/workload_test.txt. Ráfaga y llegada en ticks.

4 Gantt métricas y monitor gráfico

Cada segmento representa un intervalo semiabierto [inicio, fin). El historial agrupa intervalos consecutivos del mismo PID y usa el PID 0 para IDLE. El despacho en una frontera actualiza el estado observable, mientras que el Gantt solo registra CPU consumida. Una pausa global congela los contadores y no añade espera ni IDLE. Con el motor activo, el tiempo posterior a la última finalización sí se registra como IDLE.

Indicador	Cálculo	Disponibilidad
WT · espera	Número de ticks que el proceso pasó en READY.	Desde el alta, inicialmente 0.
TAT · retorno	Finalización menos llegada.	Al finalizar.
RT · respuesta	Primer tick ejecutado menos llegada.	Desde el primer intervalo consumido.
Ocupación de CPU	100 × ticks ocupados / reloj transcurrido.	0 % en t = 0. Incluye IDLE en el denominador.

Los promedios de WT, TAT y RT se calculan sobre procesos finalizados. Cuando ninguno ha terminado, la consola muestra N/A para esos promedios. En esta simulación sin E/S, un proceso finalizado cumple TAT = WT + ráfaga inicial. La suma de ráfagas consumidas coincide con los ticks ocupados. Ambas relaciones se comprueban en las aserciones de C++.

Monitor Qt

El monitor muestra PCB, reloj, política, quantum si corresponde, actividad, velocidad, métricas y Gantt. Con Qt habilitado, gui publica una solicitud atómica y el hilo principal inicia una segunda instancia del ejecutable con --gui-monitor mediante posix_spawnp. La vista recibe instantáneas serializadas con QDataStream por un socket Unix no bloqueante. El productor apunta a una actualización cada 100 ms y conserva un frame pendiente si el envío es parcial. El hijo lee con un temporizador de 20 ms y la interfaz repinta cada 100 ms.

El hijo mantiene el bucle de Qt al cerrar la ventana y una nueva solicitud puede mostrarla de nuevo. Un fallo de inicialización de Qt termina el hijo y el proceso principal puede seguir atendiendo la consola. En el cierre se cierra el socket y se recoge al hijo con waitpid. La compilación sin Qt conserva la CLI y devuelve un error descriptivo ante gui.

5 Validación y resultados esperados

Escenario interactivo y referencia Round Robin

Con P1, P2 y P3 disponibles desde t = 0, ráfagas 5, 3 y 2 y quantum 2, se obtiene la siguiente referencia para RR hasta el final.

Gantt RR	0–2	2–4	4–6	6–8	8–9	9–10
CPU	P1	P2	P3	P1	P2	P1

Proceso	Primera ejecución	Fin	WT	TAT	RT
P1	0	10	5	10	0
P2	2	9	6	9	2
P3	4	6	4	6	4
Promedio	—	—	**5,00**	**8,33**	**2,00**

Figura 3 y tabla. Gantt y tiempos de la referencia RR, en ticks. CPU 100 % en t = 10.

Para reproducir el escenario mixto, configurar RR 2 y 1000 ms/tick, crear los tres procesos, iniciar y consultar ps durante la alternancia. Enviar entradas inválidas y comprobar que el prompt siga disponible. Pausar, anotar el reloj con gantt y el PCB con pcb 1, elegir SRTF y reanudar. Registrar gantt y stats al finalizar, junto con el tick de cambio, para poder contrastar el resultado.

Si el cambio se hace exactamente en t = 2, P2 ya está despachado por RR pero aún no ha consumido su primer tick. Al reanudar, SRTF elige P3. La referencia calculada es P1 [0, 2), P3 [2, 4), P1 [4, 7) y P2 [7, 10).

Escenario batch con SRTF

En t = 1, Job_B llega con dos ticks pendientes y desplaza a Job_A, que conserva tres. En t = 3 termina Job_B y Job_C entra con un tick. Después de Job_C, Job_A consume sus tres ticks restantes.

Gantt SRTF	0–1	1–3	3–4	4–7
CPU	Job_A	Job_B	Job_C	Job_A

Proceso	Llegada	Primera ejecución	Fin	WT	TAT	RT
Job_A	0	0	7	3	7	0
Job_B	1	1	3	0	2	0
Job_C	3	3	4	0	1	0
Promedio	—	—	—	**1,00**	**3,33**	**0,00**

Figura 4 y tabla. Cálculo manual para la traza batch, en ticks. CPU 100 % en t = 7. Si se pausa en t = 8, el Gantt añade IDLE [7, 8) y la ocupación cae a 87,5 %.

6 Cobertura de pruebas y trazabilidad

Prueba	Comprobaciones leídas en el código
tests/tests.cpp	RR y límites del quantum, SRTF y empates, lotes fuera de orden, llegada pasada y rollback, fronteras de despacho, parser, cambio de política, invariantes y concurrencia.
tests/cli_test.py	Ejecutable real, mensajes de error, exit y EOF, CSV inválidos sin consumir PID, CRLF, Gantt y métricas batch. La secuencia inicial envía P1 y P2 sin esperar avance entre start y pause.
tests/gui_test.cpp	Widgets en modo offscreen, estado NEW, actualización de métricas y Gantt, pausa, ocultación y reapertura. El caso guarda build/gui-preview.png cuando se ejecuta.
tests/gui_cli_test.py	Solicitudes repetidas de GUI, continuidad de CLI, exit y EOF, sesión gráfica ausente y fallo del backend Qt.

El Makefile compila normalmente en C++23 con -pthread, avisos estrictos y optimización. make test ejecuta la suite C++ y la prueba Python de consola. make GUI=1 gui-test requiere Qt Widgets y ejecuta las dos pruebas de monitor. make sanitize compila y ejecuta la suite C++ con ASan y UBSan. Las pruebas Python emplean procesos y tuberías de un entorno POSIX.

7 Limitaciones y conclusiones

El modelo representa una sola CPU y un conjunto de procesos con una ráfaga conocida por proceso. Excluye E/S, memoria, coste de cambio de contexto y prioridad dinámica. SRTF puede prolongar la espera de un proceso largo si siguen llegando trabajos cortos. El retardo real depende del sistema anfitrión, por lo que las métricas describen el reloj simulado y el escenario de carga.

La arquitectura separa el análisis de comandos de la ejecución temporizada. Un mutex permite que cada consulta vea un estado consistente, y las instantáneas permiten formatear la salida fuera de la sección crítica. El despacho en la frontera del tick explica por qué una consulta puede mostrar al siguiente proceso en RUNNING mientras su ráfaga aún está intacta.

Los cálculos manuales fijan dos referencias reproducibles. Con los tres trabajos disponibles en t = 0, RR completa 10 ticks ocupados y da una espera media de 5,00 ticks. SRTF batch completa siete ticks ocupados y da una espera media de 1,00 tick. Las cargas difieren en llegadas y ráfagas, por lo que estos resultados validan cada escenario por separado. Para comparar las políticas en rendimiento sería necesario aplicar ambas a una misma traza y especificar la ventana de medición.

La evidencia revisada sustenta la estructura del simulador, las reglas de despacho y las aserciones previstas. El cierre de la validación experimental requiere conservar salidas vinculadas a la revisión de código, incluida la prueba interactiva con el tick de cambio registrado y los resultados de las suites en un entorno POSIX.

Referencias bibliográficas

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). *Fundamentos de sistemas operativos* (Vuelapluma, S. L., Trad.; 7.ª ed.). McGraw-Hill/Interamericana de España. (Obra original publicada en 2005).

Stallings, W. (1997). *Sistemas operativos* (J. M. Dodero Beardo, E. Torres Franco, & M. Katrib Mora, Trads.; 2.ª ed.). Prentice Hall. (Obra original publicada en 1995).

Tanenbaum, A. S. (2009). *Sistemas operativos modernos* (A. V. Romero Elizondo, Trad.; 3.ª ed.). Pearson Educación. (Obra original publicada en 2008).

Anexo de IA

https://chatgpt.com/share/6af3be35-23e8-83e9-a89e-cdac0869d4e4